ЛАБОРАТОРНАЯ РАБОТА №4
SCA: Trivy, анализ зависимостей и SBOM

Теория → пошаговая практика → проверка результата
2 академических часа · учебный проект vulnerable-app

Выполняем только на локальной учебной копии. Реальные секреты и чужие системы не используем.

1. Что вы изучите

Научиться находить уязвимые зависимости, читать таблицу Trivy, создавать SBOM и принимать решение с учётом реального использования компонента.

• Запустите Trivy fs.

• Разберёте installed и fixed version.

• Создадите CycloneDX SBOM.

• Оцените контекст legacy-компонента.

• Предложите обновления и quality gate.

Опыт программирования не требуется: команды и фрагменты кода можно копировать. После каждого шага объясняется, что должно получиться и как проверить результат.

2. Теория простыми словами

Что такое зависимость

Проект использует чужие пакеты. Прямая зависимость записана в requirements.txt. Транзитивная приходит вместе с другой библиотекой. Уязвимость может находиться не в вашем коде, а в коде пакета.

Что делает SCA

Software Composition Analysis определяет компоненты и сопоставляет версии с базами уязвимостей. Trivy читает lockfiles и requirements, анализирует образы и сообщает установленную и исправленную версии.

Что такое CVE и CVSS

CVE — идентификатор известной уязвимости. CVSS — оценка технической тяжести. Риск проекта зависит ещё и от использования функции, доступности из сети, прав и наличия exploit. High не всегда означает немедленную блокировку любого релиза.

Зачем нужен SBOM

Software Bill of Materials — список компонентов артефакта. Если завтра выйдет новая уязвимость, SBOM поможет быстро понять, какие релизы её содержат. SBOM сам не находит уязвимости.

Почему legacy-компонент отдельный

Файл legacy-component/requirements.txt намеренно содержит старые пакеты для анализа. Он не устанавливается и не входит в runtime нашего Flask-приложения. Это нужно доказать в отчёте, а не просто написать «не опасно».

Обновление — не только команда

После изменения версии нужны тесты совместимости. Если fixed version отсутствует, применяют компенсирующие меры, ограничивают exposure, назначают владельца и отслеживают выход

исправления.

3. Перед началом

• Распакуйте практикум и откройте терминал в каталоге vulnerable-app.

• Нужны Python 3.11+, браузер и терминал. Для ZAP и альтернативных запусков — Docker Desktop.

• Команды показаны для Git Bash. Для PowerShell используйте .venv\Scripts\Activate.ps1 вместо source.

• Создайте папку reports для результатов. Не публикуйте отчёты, пока не проверили их содержимое.

• Сохраняйте исходную копию проекта, чтобы сравнивать состояние до и после исправлений.

4. Практика по шагам

Шаг 1. Запустить Trivy по файловой системе

Зачем: Получим исходный список уязвимостей и секретов. Не устанавливайте legacy requirements.

Что сделать:

1. Откройте терминал в vulnerable-app и создайте reports.

2. Запустите Trivy fs с сохранением JSON.

3. Если Trivy не установлен, используйте Docker.

4. Зафиксируйте версию инструмента: база уязвимостей меняется со временем.

Команда или код:

```
mkdir -p reports
trivy fs --scanners vuln,secret --format json --output reports/trivy-fs.json .

# Docker-вариант:
docker run --rm -v "${PWD}:/src" aquasec/trivy:latest fs --scanners vuln,secret /src
```

Ожидаемый результат: В summary есть legacy-component/requirements.txt и backend/config.py. Количество CVE может отличаться после обновления базы — это нормально.

Шаг 2. Прочитать таблицу уязвимостей

Зачем: Каждая строка должна превращаться в понятное действие.

Что сделать:

1. Найдите Django 2.0.9, requests 2.19.0, urllib3 1.24.1 и Jinja2 2.10.

2. Для четырёх CVE выпишите Library, Vulnerability, Severity, Installed и Fixed Version.

3. Прочитайте короткое описание и ссылку AVD/NVD.

4. Выберите действие: обновить, проверить контекст или применить компенсацию.

Пример и объяснение: requests 2.19.0 → CVE-2018-18074 → fixed 2.20.0. Проблема: Authorization header при небезопасном redirect. Действие: обновить и проверить сценарии HTTP-клиента.

Ожидаемый результат: Есть таблица минимум из четырёх находок с версиями и обоснованным действием.

Шаг 3. Создать и открыть SBOM

Зачем: Нужно увидеть не список уязвимостей, а состав компонентов.

Что сделать:

1. Создайте CycloneDX в reports.

2. Откройте JSON и найдите bomFormat и components.

3. Сопоставьте компоненты с requirements.

4. Опишите, как этот файл поможет после выпуска новой CVE.

Команда или код:

```
trivy fs --format cyclonedx --output reports/sbom.cdx.json .

# Docker:
docker run --rm -v "${PWD}:/src" aquasec/trivy:latest fs --format cyclonedx --output
  /src/reports/sbom.cdx.json /src
```

Ожидаемый результат: Файл sbom.cdx.json существует и содержит компоненты. В нашем контрольном запуске их было 9; количество может меняться в зависимости от цели.

Шаг 4. Оценить риск в контексте

Зачем: Нельзя принимать решение только по цвету severity.

Что сделать:

1. Проверьте, импортируется ли Django, requests, urllib3 или Jinja2 из legacy в backend.

2. Зафиксируйте, что основной runtime задаётся корневым requirements.txt.

3. Опишите риск уязвимого компонента для production, для CI и для учебного анализа.

4. Выберите одну находку и объясните достижимость уязвимой функции.

Пример и объяснение: Вывод должен звучать так: «Пакет обнаружен в legacy-component. В runtime не включён, потому что backend импортирует Flask и не использует этот манифест. Действие: обновить учебный манифест и повторить сканирование».

Ожидаемый результат: В отчёте явно указано: legacy используется только как учебный манифест, поэтому находки реальны для него, но не автоматически для работающего Flask-приложения.

Шаг 5. Предложить исправления зависимостей и Dockerfile

Зачем: Исправление должно устранять причину и проверяться повторно.

Что сделать:

1. Для выбранных пакетов запишите fixed version из отчёта.

2. Предложите обновление до совместимой версии, а не просто последней доступной.

3. Перечислите тесты после обновления.

4. Запустите trivy config и найдите DS-0002: контейнер без USER.

5. Предложите добавление непривилегированного пользователя в Dockerfile.

Команда или код:

```
trivy config .

# Пример для Dockerfile после установки пакетов:
RUN useradd --create-home labuser && chown -R labuser:labuser /app
USER labuser
```

Ожидаемый результат: Есть конкретный план обновления и пример создания пользователя. Повторный Trivy после реального изменения должен подтвердить результат.

Шаг 6. Сформулировать quality gate

Зачем: Политика помогает одинаково принимать решения в CI.

Что сделать:

1. Определите, какие новые находки блокируют сборку.

2. Укажите отдельное правило для production и test-only компонентов.

3. Опишите действие при отсутствии fixed version и при ошибке сканера.

4. Добавьте владельца, срок и дату пересмотра исключения.

Пример и объяснение: Новый Critical в production с fixed version: блокировать PR, обновить и повторить Trivy. Ошибка загрузки базы: проверка считается невыполненной, а не «чистой».

Ожидаемый результат: Получилась короткая таблица: сигнал → действие → владелец → повторная проверка.

5. Что сдать в отчёте

• Команда, версия Trivy и summary.

• Таблица четырёх CVE с installed/fixed.

• SBOM и объяснение его назначения.

• Контекст legacy и план обновлений.

• DS-0002 и quality gate.

В конце добавьте краткий вывод: какой дефект был найден, какая причина устранена, что проверено повторно и какие риски остались.

6. Проверь себя

1. Чем SCA отличается от SAST?

2. Чем SBOM отличается от отчёта CVE?

3. Почему CVSS не равен риску проекта?

4. Что делать при отсутствии fixed version?

5. Почему после обновления нужны тесты?

7. Критерии оценивания

Критерий	Баллы
Запуск и таблица CVE	25
SBOM	15
Контекст риска	20
Исправления	20
Quality gate и отчёт	20

Максимум — 100 баллов. За использование чужой системы без разрешения или публикацию настоящих секретов работа не принимается. Честно описанный отрицательный результат и ограничения инструмента являются нормальной частью анализа.

8. Источники

• ГОСТ Р 56939-2024: https://www.gostinfo.ru/catalog/Details/?id=8094263

• ГОСТ Р 71207-2024: https://protect.gost.ru/gost/details/9dd6f064-6014-4416-a3ca-ef72ce0a6c3a

• OWASP ASVS: https://owasp.org/www-project-application-security-verification-standard/

• Trivy: https://trivy.dev/latest/docs/target/filesystem/

• CycloneDX: https://cyclonedx.org/